

MNC: A multi-agent framework for complex network configuration

Cui Wang^a, Huan Li^{b,*}

^a The 54th Research Institute of China Electronics Technology Group Corporation, No. 589 Zhongshan Street, Shijiazhuang, 050081, HeBei, China

^b Hebei University Of Science and Technology, No. 26 Yuxiang Street, Shijiazhuang, 050081, HeBei, China

ARTICLE INFO

Keywords:

Complex network configuration
Multi-agent
Large language model

ABSTRACT

Recent progress in large language models (LLMs) has led to substantial improvements in their ability to perform a wide range of natural language processing tasks, particularly in handling complex scenarios. These models exhibit strong generalization capabilities and reasoning skills, making them well-suited for tasks that require integrating external knowledge and logical reasoning. In this paper, we introduce Multi-agent based Network Configuration (MNC), a novel multi-agent framework designed to leverage LLMs for complex network configuration tasks. Our framework consists of three core components: (1) the **Requirement Analysis Module**, which interprets user queries and retrieves relevant external network configuration knowledge; (2) the **Configuration Generation Module**, which uses an iterative Chain-of-Thought (COT) approach to produce and refine multiple analysis pathways; and (3) the **Configuration Refinement Module**, which evaluates and improves the final network configuration through a reflection-driven mechanism. We evaluate MNC on a network configuration dataset, where our proposed MNC outperforms existing baseline methods. Furthermore, an ablation study demonstrates the individual contributions of each module to the framework's overall effectiveness. This research underscores the potential of LLM-based systems to advance complex network configuration tasks.

1. Introduction

The rapid advancement of language model technology has led to the emergence of large language models (LLMs), which have shown impressive capabilities across a wide range of natural language processing (NLP) tasks. LLMs have consistently surpassed traditional methods in well-established NLP tasks such as text classification (Joulin, Grave, Bojanowski, & Mikolov, 2016; Zhang, Zhao, & LeCun, 2015), summarization (Chen et al., 2018; Gao et al., 2018), question answering (Gao et al., 2019, 2021), and information extraction (Peng, Wang, Yao, Wang, & Shang, 2024). A key strength of LLMs lies in their strong generalization ability, enabling them to perform well on unseen data with minimal reliance on domain-specific knowledge. Moreover, LLMs have made significant progress in handling complex, multi-step tasks (Wang et al., 2024). A notable example is Retrieval-Augmented Generation (RAG) (Gao et al., 2023), an active area of research that requires LLMs to integrate external knowledge, perform semantic matching, extract relevant information, and generate coherent responses. This demonstrates that LLMs are capable of handling tasks that go beyond simple, single-step operations and require reasoning and information integration. These developments highlight the potential of LLMs to tackle increasingly complex problems across various domains.

The widespread availability of data requires strong analysis methods in many fields. Generating effective network configurations goes beyond simply identifying trends; it requires combining domain knowledge and practical experience. This combination helps engineers uncover complex patterns in the data, revealing key insights and guiding actionable solutions. As a result, integrating domain knowledge is essential for improving the quality and impact of network configuration generation efforts.

The complexity of network configuration generation demands a multi-step approach to produce accurate and efficient configurations, presenting significant challenges for current LLMs. This paper identifies three critical stages in effective network configuration generation: (1) comprehensive requirement analysis through systematic examination of user queries and network specifications, which forms the foundation for configuration design; (2) configuration pattern discovery through the integration of external network knowledge to contextualize requirements, explain current network behaviors, and predict optimal configurations within a logical framework; and (3) synthesis of network insights and external knowledge to generate meaningful configurations that ensure proper functionality, scalability, and security. This configuration generation process highlights that effective network

* Corresponding author.

E-mail addresses: wang_99881@163.com (C. Wang), lhbawy@gmail.com (H. Li).

<https://doi.org/10.1016/j.iswa.2025.200531>

Received 4 April 2025; Received in revised form 19 April 2025; Accepted 3 May 2025

Available online 17 May 2025

2667-3053/© 2025 The Authors. Published by Elsevier Ltd. This is an open access article under the CC BY-NC-ND license (<http://creativecommons.org/licenses/by-nc-nd/4.0/>).

design requires not only technical expertise but also sophisticated reasoning skills. While LLMs have demonstrated significant progress in acquiring various cognitive abilities, their performance in complex network configuration tasks remains limited. The primary challenge lies in their inability to effectively decompose complex configuration tasks, a crucial requirement for successful network design that typically involves task decomposition, iterative refinement, and reflective evaluation of results. Motivated by these observations, this paper explores the potential of LLMs in addressing complex network configuration challenges. Our research focuses on developing methodologies to enhance LLMs' capabilities in task decomposition and configuration reasoning, aiming to bridge the gap between current LLM capabilities and the requirements of sophisticated network configuration tasks.

In this paper, we introduce **Multi-agent based Network Configuration (MNC)**, a multi-agent framework designed for network configuration generation using large language models. The framework consists of three core modules: **(1) Requirement Analysis Module:** This module interprets user queries related to network configuration and identifies key requirements. It analyzes the fundamental aspects of the network setup and leverages external knowledge retrieval tools to gather relevant configuration guidelines and best practices. This supporting knowledge is crucial for generating accurate and efficient network configurations. **(2) Configuration Generation Module:** This module produces an initial network configuration through an iterative process. *Initial Generation:* The LLM employs a chain-of-thought (COT) approach to generate multiple distinct configuration pathways. *Self-Consistency Evaluation:* The consistency of these pathways is assessed to identify and mitigate potential errors in individual configurations. *Iterative Refinement:* If inconsistencies are detected, a multi-agent debate mechanism is used to critically evaluate the existing pathways. The LLM then generates new configuration pathways based on insights from these debates. This process continues until a consistent and reliable configuration is achieved. **(3) Configuration Refinement Module:** A quality assessor evaluates the generated configuration pathways against predefined criteria to select the most robust one. Inspired by the "critic" method, the chosen configuration undergoes further refinement through constructive feedback. Finally, the response enhancement module integrates this feedback and the best COT result to produce the final network configuration.

To evaluate the performance of our proposed method, we conducted experiments on a network configuration dataset derived from real-world enterprise network scenarios. Such datasets are particularly valuable as they reflect practical network configurations, ensuring the relevance and reliability of our study. Additionally, the structured nature of network configuration data allows for systematic analysis and the identification of key patterns and dependencies. We also conduct experiments to verify the correctness of the generated configuration on a widely-used network simulator. We compared our approach with several baseline methods to demonstrate its effectiveness in generating accurate and efficient network configurations. Furthermore, we conducted an ablation study to analyze the individual contributions of each module in our framework, highlighting their roles in improving the overall performance of the system. This comprehensive evaluation underscores the robustness and practical applicability of our method in real-world network configuration tasks.

To summarize, our contributions are as follows:

1. We propose **Multi-agent based Network Configuration (MNC)**, a novel multi-agent framework based on large language models specifically designed to address complex network configuration tasks.
2. We develop an iterative configuration generation process that produces multiple configuration pathways using a chain-of-thought approach. These pathways are evaluated and refined to ensure robust and reliable network configurations.
3. To enhance the quality of the final configuration, we introduce a refinement module that critically evaluates the selected configuration pathway. This module incorporates feedback and optimizations to generate a superior network configuration.
4. We conduct extensive experiments on a real-world network configuration dataset and network simulator, demonstrating that our method outperforms existing baseline approaches. Additionally, an ablation study confirms the individual contributions of each module to the overall effectiveness of the framework.

2. Related work

This section provides an overview of research areas closely related to our work. We examine advancements in multi-agent systems built on large language models, techniques for integrating external tools with LLMs, and approaches for network configuration analysis. These directions collectively inform the design and implementation of our proposed framework for addressing complex network configuration tasks.

2.1. LLM-based multi-agent framework

In recent years, LLM-based multi-agent frameworks have gained considerable attention for their potential in enabling collaborative intelligence and efficient task execution (Han et al., 2024; Ishibashi & Nishimura, 2024; Liu et al., 2024). These frameworks not only enhance automation systems but also provide innovative solutions for tackling complex collaborative tasks. By enabling multiple agents to work together, they achieve higher efficiency in problem-solving across diverse domains. The ability of agents to communicate and collaborate allows for a more comprehensive approach to addressing complex challenges, particularly in dynamic environments where integrating multiple perspectives is crucial. This makes multi-agent systems particularly well-suited for tasks such as network configuration, where coordination and adaptability are essential.

2.1.1. General agent methods

General agent methods focus on enabling agents to collaborate effectively to solve complex tasks. For instance, Generative Agents (Hong et al., 2024) proposed a framework where agents simulate interactions by generating behaviors, allowing them to complete tasks in open environments. This approach has proven effective in open-world scenarios that demand dynamic decision-making and adaptability. Similarly, MetaGPT (Hong et al., 2023) introduced an automated software development framework by assigning specific roles to each agent, enabling collaborative workflows to achieve development objectives. The structured pipeline of agent interactions enhances productivity and task efficiency. Additionally, Multi-agent debate (Chan et al., 2023) introduced collaboration and debate mechanisms among agents, improving decision-making and task execution in multi-agent systems. The debate structure in these frameworks encourages diverse perspectives and reasoning, leading to stronger final outcomes by incorporating multiple viewpoints. These advancements highlight the potential of multi-agent systems to address complex tasks through coordinated and adaptive collaboration.

2.1.2. Critic methods

Critic methods are a widely used strategy evaluation mechanism in reinforcement learning, providing feedback on agents' actions to refine and optimize their strategies. These methods are particularly valuable in multi-agent systems, where agents must adapt to dynamic and evolving environments to make informed decisions. For example, Multi-Agent debate (Chan et al., 2023) incorporates critic mechanisms by enabling agents to engage in structured debates, ensuring that diverse perspectives are considered during task execution. This approach fosters critical thinking, reduces bias, and enhances the overall quality

of decisions. Similarly, self-play (Cao et al., 2024; Wu et al., 2024) introduces a reflection process that evaluates the reasoning behind agents' decisions rather than simply aggregating multiple viewpoints, as in traditional critic methods. By focusing on the quality of reasoning, this method improves decision-making, particularly in complex, multi-step tasks where conventional approaches may struggle. These advancements demonstrate the importance of critic mechanisms in enhancing the adaptability and effectiveness of multi-agent systems.

2.1.3. Chain-of-thought methods

Chain-of-Thought (CoT) methods focus on guiding models to generate step-by-step reasoning processes to tackle complex problems. CoT (Wei et al., 2022) showcased the ability of models to handle intricate tasks by breaking them down into sequential reasoning steps, enabling more structured problem-solving. Subsequent advancements, such as Zero-shot-CoT (Kojima, Gu, Reid, Matsuo, & Iwasawa, 2022) and Least-to-Most prompting (Zhou et al., 2023), further enhanced the model's capability to perform zero-shot learning and incremental reasoning. These approaches allow models to address tasks without task-specific training, making them highly versatile in new and unfamiliar scenarios. Additionally, Self-Consistency (Chen et al., 2023) introduced the idea of generating multiple reasoning chains and selecting the most consistent answer, improving the accuracy and robustness of the model's outputs. ReAct (Aksitov et al., 2023; Yao et al., 2023) expanded on CoT by combining reasoning with action, enabling models to dynamically adapt their reasoning processes based on interactions with external environments. This approach emphasizes iterative refinement through feedback, enhancing the reliability of conclusions and enabling more adaptive and effective problem-solving. These developments highlight the potential of CoT-based methods to improve reasoning and decision-making in complex tasks.

2.2. Retrieval-augmented generation

Retrieval-augmented generation (RAG) (Gao et al., 2023) emphasizes the integration of knowledge retrieval systems with generative models. RAG enables models to access and incorporate relevant information during the generation process, improving the accuracy and depth of their outputs. This approach not only expands the model's knowledge base but also ensures more reliable and contextually appropriate responses by leveraging real-time information from external sources. In the context of CoT-based tool learning, frameworks such as tool learning for EDA (Pu, He, Qiu, Wu, & Yu, 2024) and IRCot (Trivedi, Balasubramanian, Khot, & Sabharwal, 2023) have explored the combination of reasoning steps and knowledge retrieval. tool learning for EDA enhances the reliability and fidelity of generated explanations by retrieving relevant knowledge based on the reasoning steps derived from CoT. Similarly, IRCot interleaves reasoning and retrieval steps, enabling more dynamic and context-aware reasoning processes. This approach allows the model to adapt its reasoning based on external knowledge, improving both the depth and accuracy of its conclusions. These advancements demonstrate the potential of integrating retrieval mechanisms with reasoning frameworks to enhance the performance of generative models in complex tasks.

2.3. Tool learning

Tool learning is a crucial research area aimed at enhancing the capabilities of models by enabling them to utilize external tools, thereby extending their functionality beyond their inherent limitations. This approach involves equipping LLMs with the ability to interact with external resources, such as APIs, databases, or knowledge retrieval systems, to augment their internal knowledge and improve task performance. This represents a significant advancement in model design, as it expands the range of tasks LLMs can effectively handle.

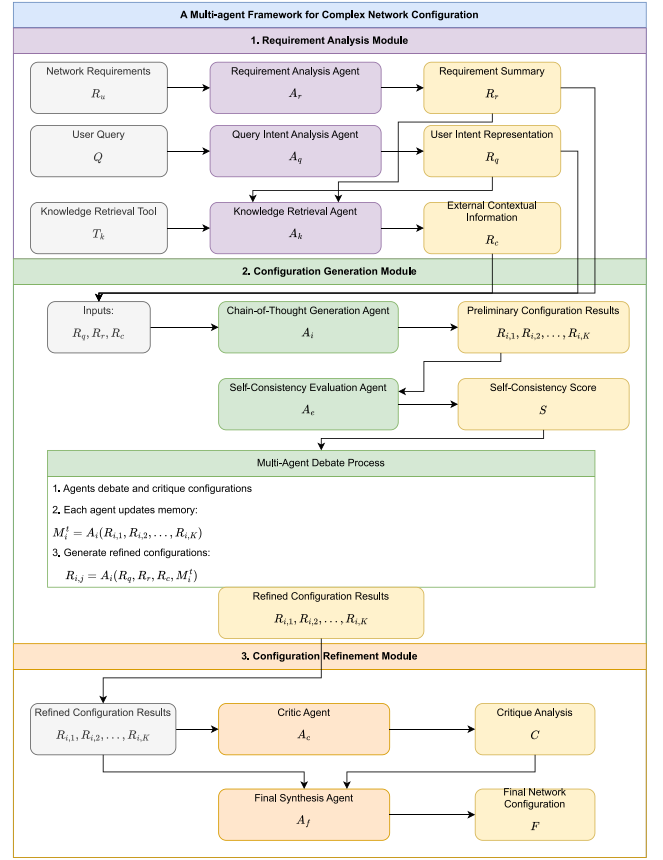


Fig. 1. Overall architecture of our proposed Multi-agent based Network Configuration (MNC).

Recent developments have highlighted the effectiveness of tool learning in boosting model performance. For instance, Confucius (Gao et al., 2024) introduces an iterative tool learning framework, where LLMs refine their ability to use external tools through feedback and progressively challenging tasks. This method encourages continuous improvement in tool usage, enabling models to tackle increasingly complex problems. The iterative process not only enhances performance but also promotes adaptability to new tool functionalities, leading to more efficient problem-solving. StableToolBench (Guo et al., 2024) adopts a different strategy by leveraging retrieval mechanisms to revise and improve the accuracy of input text. Similarly, Toolformer (Schick et al., 2023) advances tool learning by incorporating reinforcement learning techniques, allowing the model to autonomously determine when and which tools to use based on task requirements. ToolACE-R (Zeng et al., 2025) further enhances adaptability through a self-refinement mechanism, enabling iterative optimization of tool invocations without external feedback, while START (Li et al., 2025) integrates tool-augmented reasoning with self-learning techniques like hint-infer to mitigate hallucinations in complex tasks. These advancements underscore the transformative potential of tool learning in enhancing the capabilities of LLMs.

3. MNC

In this section, we present the details of our proposed method, Multi-agent based Network Configuration (MNC), with its architecture illustrated in Fig. 1. Our MNC framework consists of three core modules:

- **Requirement Analysis Module:** This module carefully processes user-provided network requirements R and configuration queries

Q . It extracts relevant contextual information from external knowledge sources to ensure a thorough understanding of the user's needs and the specific network environment. This step lays the foundation for generating accurate and tailored configurations.

- **Configuration Generation Module:** This module drives the core configuration generation process. Building on the insights from the Requirement Analysis Module, it produces an initial network configuration. To ensure reliability, we employ a Self-Consistency Measurement to identify and address potential errors, followed by an Iterative Refinement process to progressively improve the quality of the generated configuration.
- **Configuration Refinement Module:** This module acts as a critical evaluator, rigorously assessing the outputs of the Configuration Generation Module. It incorporates a "critic" mechanism to identify areas for improvement and integrates feedback to refine the configuration. This iterative refinement process ensures the final configuration is robust, efficient, and aligned with the user's requirements.

The seamless integration of these three modules enables MNC to effectively tackle complex network configuration tasks, delivering reliable and optimized solutions.

3.1. Requirement analysis module

To generate a high-quality network configuration, it is essential to first understand the user's requirements and the specific characteristics of the network environment (Chen et al., 2019; Han et al., 2019; Zhao et al., 2019), such as performance constraints, security needs, and scalability goals. Therefore, we begin by utilizing a large language model to analyze the user-provided requirements R_u and extract key insights:

$$R_r = A_r(R_u), \quad (1)$$

where A_r represents the agent responsible for interpreting user requirements and identifying critical network parameters, and R_r is a textual output summarizing the user's needs and the essential features of the network environment. The instruction provided to agent A_r would be as follows:

You are a network configuration expert. The user has provided a network requirement and some network knowledge documents. Analyze the user query and extract key knowledge that is relevant to the user query.

To ensure the generated network configuration meets the user's requirements, we first analyze the user's query Q to extract their core intent and objectives. We employ a large language model-based agent A_q to perform this user intent extraction:

$$R_q = A_q(Q) \quad (2)$$

where A_q represents the agent responsible for analyzing user intent, and R_q is a set of keywords or structured outputs capturing the core intent and objectives derived from the user query Q . The instruction provided to agent A_q would be as follows:

Analyze the user's queries, identify the main context, objectives, and specific requirements mentioned in the query, and extract a set of keywords or structured outputs that accurately represent the user's core intentions and network configuration needs.

To generate a comprehensive and accurate network configuration, relying solely on the user-provided requirements R_u is often insufficient. Network configuration tasks typically require additional contextual knowledge, such as industry standards, best practices, and technical guidelines. Understanding these nuances is essential for creating configurations that are both functional and optimized. Inspired by the capabilities of large language models, we first construct a knowledge retrieval tool T_k . We then combine this tool with the previously extracted user intent representation R_q and the network requirement summary R_r , feeding them into a knowledge retrieval agent A_k . During the process of using retrieval tools for knowledge retrieval, we utilize the user intent representation R_q and the network requirement summary R_r as inputs to the retrieval tool. Based on the semantic similarity of the returned knowledge documents, the most relevant N documents are selected as R_c . This agent analyzes the semantic relationship between the user's intent and the network requirements, generates relevant search queries, and retrieves external contextual information from trusted sources:

$$R_c = A_k(R_q, R_r, T_k) \quad (3)$$

where R_c represents the external contextual information retrieved from knowledge sources. The instruction provided to agent A_k would be as follows:

Use search tools to retrieve external knowledge based on the user's intent keywords and network requirement summaries, obtain relevant contextual information, organize and refine the search results, and ensure that the retrieved information is accurate, clear, and directly applicable to the network configuration task.

3.2. Generation module

3.2.1. Initial generation

Inspired by the effectiveness of chain-of-thought prompting, we employ an agent A_i based on this principle to conduct an initial network configuration analysis:

$$R_{i,j} = A_i(R_q, R_r, R_c) \quad (4)$$

where $R_{i,j}$ represents the j th preliminary network configuration result. In this approach, we introduce a novel chain-based configuration generation method. Agent A_i utilizes chain-of-thought prompts to explore network requirements and constraints in greater detail, producing logically structured and insightful initial configurations. This chain-based methodology enables the agent to dissect network requirements more comprehensively and generate more fine-grained and coherent configuration outputs.

Within our framework, we perform K distinct iterations of chain-based configuration analysis, yielding K unique preliminary configuration results: $R_{i,1}, R_{i,2}, \dots, R_{i,K}$. These diverse initial configurations provide a broader spectrum of perspectives on the network requirements, enriching the subsequent refinement process.

3.2.2. Self-consistency measurement

After obtaining K distinct preliminary configuration results, we recognize that these configurations may vary in quality, with some potentially containing errors or inefficiencies. Leveraging the principles of Self-Consistency, we propose a novel Self-Consistency Measurement method tailored for network configuration tasks. To assess the consistency of these preliminary configurations, we evaluate them across five key aspects:

- **Relevance:** How effectively does the configuration address the user's original requirements and constraints?

- **Clarity:** Is the configuration presented in a clear, concise, and understandable manner?
- **Completeness:** Does the configuration provide sufficient detail and context to support its implementation?
- **Accuracy:** Is the configuration technically correct and aligned with best practices?
- **Insightfulness:** Does the configuration offer unique or innovative solutions to the network requirements?

These five aspects serve as comprehensive rules for evaluating the quality and consistency of each preliminary configuration. In this method, these five criteria are assigned equal weights, with each criterion accounting for 20% of the total weight. This design of evenly distributed weights ensures that each criterion contributes equally to the final self-consistency score. Such an approach guarantees that all evaluation criteria influence the final score without bias, thereby ensuring the comprehensiveness and balance of the scoring results. Specifically, we score the performance of each criterion and then calculate a composite self-consistency score by summing these scores with equal weighting. This score not only reflects the quality of the configuration but also effectively assesses its consistency across multiple dimensions. By aggregating scores across these criteria, we arrive at a holistic measure of their overall coherence and reliability. We employ a self-consistency evaluation agent A_e to measure the self-consistency:

$$S = A_e(R_{i,1}, R_{i,2}, \dots, R_{i,K}) \quad (5)$$

where $S \in \mathbb{R}$ is the self-consistency score, and the instruction of agent A_e is:

Receive the preliminary network configuration results and evaluate the quality and consistency of each configuration. Perform detailed assessments based on the following five key aspects:

Relevance: Evaluate whether the configuration effectively addresses the user's original requirements and constraints.

Clarity: Assess whether the configuration is presented clearly, concisely, and understandably.

Completeness: Determine whether the configuration provides sufficient detail and context to support its implementation.

Accuracy: Verify whether the configuration is technically correct and aligned with best practices.

Insightfulness: Evaluate whether the configuration offers unique or innovative solutions to the network requirements.

Assign scores for each configuration based on the above criteria and aggregate them to compute an overall self-consistency score. Ensure the scoring process is transparent and logically rigorous, documenting the rationale for each score. Return the final self-consistency score along with a brief evaluation summary to support further decision-making or improvements.

When the self-consistency measurement reveals a low level of coherence among the K preliminary configurations, we introduce a multi-agent debate method. This approach fosters an environment of intellectual discourse by having the very agents responsible for generating these configurations engage in a structured debate. Through this simulated debate, the agents critically examine each other's reasoning, identify inconsistencies and potential errors, and refine their understanding of the network requirements based on the collective insights

gleaned from the discussion. This collaborative process aims to elevate the quality and accuracy of the overall network configuration by leveraging the strengths of multiple perspectives.

3.2.3. Iterative refinement

During the multi-agent debate, the agents generating the network configurations engage in a constructive discussion, pooling their insights and refining their understanding. This process allows them not only to absorb the strengths of other agents' configurations but also to identify and rectify their own potential errors. Following the debate, each agent summarizes the key takeaways, forming an updated memory M'_i :

$$M'_i = A_i(R_{i,1}, R_{i,2}, \dots, R_{i,K}) \quad (6)$$

where M'_i represents the i th agent's consolidated memory after the i th round of debate, and the instruction of agent A_i is:

Participate in a multi-agent debate by engaging with the configurations generated by all agents. Analyze the reasoning and solutions presented by other agents, identifying strengths and novel perspectives. Detect inconsistencies, errors, or gaps in reasoning, both in others' configurations and your own. Share your unique perspectives and insights to enrich the overall discussion. Refine your understanding by integrating valuable feedback and observations from the debate. Consolidate relevant insights and improvements derived from the debate. Update your own reasoning, solutions, and approach to reflect a more comprehensive and accurate understanding.

With this enriched memory, agent A_i generates an updated network configuration:

$$R_{i,j} = A_i(R_q, R_r, R_c, M'_i). \quad (7)$$

This iterative refinement cycle continues until the configurations generated by the agents reach a satisfactory level of consistency. At this point, we conclude the iterative configuration process, confident in the robustness and accuracy of the final results.

3.3. Configuration refinement module

To further enhance the quality of the generated network configurations, we incorporate a critic method. An agent A_c evaluates the final outputs from each of the K agents:

$$C = A_c(R_{i,1}, R_{i,2}, \dots, R_{i,K}) \quad (8)$$

where C is the detailed critique that highlights areas for improvement. The agent A_c is guided by instructions designed to assess the strengths and weaknesses of each configuration:

Evaluate the final configurations provided by the agents, carefully analyzing their reasoning, solutions, and overall quality. Identify aspects of the configuration that are particularly well-executed, highlight areas that require improvement, determine whether the configuration aligns with the user's original requirements and remains coherent across different components, evaluate whether the configuration provides actionable or meaningful outcomes for the user's needs, and provide a detailed critique for each configuration to guide further optimization for high-quality results.

Finally, we synthesize the critic's evaluation with all K network configurations to arrive at the final result:

$$F = A_f(R_{i,1}, R_{i,2}, \dots, R_{i,K}, C) \quad (9)$$

where F is the refined final network configuration, and the instruction used by agent A_f is:

Identify and preserve the most robust and insightful aspects of each preliminary configuration. Use the critique to rectify errors, clarify ambiguities, and fill in gaps in reasoning or completeness. Harmonize the integrated results to create a consistent and unified configuration, avoiding redundancy or conflict between solutions. Present the refined configuration in a clear, concise, and logically structured format, ensuring it aligns with the user's requirements.

4. Experimental setup

4.1. Research questions

To validate the effectiveness of the network configuration framework we proposed, we designed a series of experiments using network simulation tools and real-world scenarios to address the following research questions:

1. **Basic LAN Configuration and Connectivity:** How can a small local area network (LAN) be designed and configured using a Cisco 2960 switch to ensure connectivity between two PCs? What are the key steps and commands required to set up and test this basic network?
2. **Switch Port Configuration and Management:** What are the necessary steps and commands to configure switch port settings, including speed, duplex mode, and device naming? How can switch information such as version, running configuration, and port details be retrieved and managed?
3. **Remote Management via Telnet:** How can a switch be configured to allow remote management via Telnet? What are the essential configurations, including IP address assignment, user authentication, and password management, to enable secure and efficient remote access?
4. **Link Aggregation and Redundancy:** How can port aggregation be configured to achieve bandwidth expansion and redundancy between two switches? What commands and steps are required to verify the success of the link aggregation setup and ensure proper communication between devices?
5. **VLAN Configuration and Isolation:** How can VLANs be configured on a switch to isolate traffic between different departments (e.g., finance and sales) while allowing communication within the same VLAN? What are the necessary commands to set up VLANs and Trunk ports?
6. **Layer 3 Switching and Inter-VLAN Routing:** How can a Layer 3 switch be configured to enable routing between VLANs? What are the key steps to configure VLAN interfaces (SVIs) and ensure inter-VLAN communication?
7. **Multi-Switch VLAN and Trunk Configuration:** How can VLANs and Trunk ports be configured across multiple switches (Layer 2 and Layer 3) to enable communication between devices in different VLANs? What are the essential commands and configurations?

Rapid Spanning Tree Protocol (RSTP) Configuration: How can RSTP be configured on switches to prevent loops and ensure redundancy in a network with multiple links? What are the necessary commands to enable RSTP and verify its operation?

8. **Router Configuration and Remote Access:** How can a router be configured for basic management, including IP address assignment, Telnet access, and password setup? What are the steps to configure router interfaces and verify connectivity?
9. **Inter-VLAN Routing with a Router and Switch:** How can a router and switch be configured to enable communication between devices in different VLANs? What are the necessary steps to configure subinterfaces, VLANs, and Trunk ports to achieve inter-VLAN routing?

4.2. Baselines

Since network configuration tasks are comprehensive and complex, there are few existing baseline methods capable of fully addressing these tasks. Given the task-solving capabilities of LLMs, we use several state-of-the-art LLMs as baseline comparison methods. In our experiments, we selected three of the currently best-performing large language models: GPT-4o mini. Additionally, we include the open-source LLMs, llama-3.3-70b-instruct and deepseek-v3, as baselines to provide a broader comparison. For the baseline method experiments, we used the same instructions and evaluation criteria as those proposed in this paper to ensure fairness and consistency across all experiments.

4.3. Metrics

To evaluate the quality of the generated network configurations, we employed three standard metrics: Accuracy (ACC), F1-Score, and BLEU.

1. **Accuracy (ACC):** This metric measures the correctness of the generated network configurations by evaluating whether the configurations, when applied, produce the expected network behavior. It is calculated as the ratio of correctly configured tasks to the total number of tasks.
2. **F1-Score:** This metric balances precision and recall, providing a comprehensive measure of the model's ability to generate accurate and complete network configurations. It is particularly useful for evaluating the correctness of specific configuration tasks, such as VLAN setup or port aggregation.
3. **BLEU (Papineni, Roukos, Ward, & Zhu, 2002):** This metric evaluates the similarity between the generated configurations and standard reference configurations. It measures how closely the generated text matches the expected configuration commands, providing insight into the model's ability to produce syntactically and semantically correct configurations.

We also compared the time consumed by GPT-based methods in completing tasks. It should be noted that this time includes the network latency for local code to request server APIs, measured in seconds. These metrics collectively provide a robust evaluation framework, ensuring that the generated configurations are not only functionally correct but also align with industry-standard practices.

4.4. Implementation detail

In this paper, we utilize the Bocha Web Search API as the implementation of our search tool. This advanced search tool, provided by Bocha AI¹, offers developers real-time web-based search capabilities. It supports access to a wide range of online resources through its API, enabling users to quickly retrieve relevant information from the internet and delivering highly accurate results via its powerful search engine. The Bocha Web Search API optimizes the process of information retrieval and data extraction through efficient query algorithms, supporting flexible search functionalities such as text queries,

¹ <https://open.bochaai.com/>.

Table 1

Performance comparison between MNC and baseline based on GPT 4o-mini.

	ACC	F1-score	BLEU	Tokens	Code length	Time
RQ1 4o-mini	45.71	52.00	51.67	28	13	1.7777
RQ1 MNC	51.43	38.71	53.62	106	70	72.4894
RQ2 4o-mini	58.97	64.00	67.04	52	25	2.5157
RQ2 MNC	61.54	30.19	51.67	255	39	42.538
RQ3 4o-mini	48.98	63.00	52.13	31	15	1.1091
RQ3 MNC	65.31	48.48	53.34	92	71	35.3638
RQ4 4o-mini	61.54	61.00	51.67	86	43	2.6119
RQ4 MNC	69.23	45.38	52.95	330	115	46.062
RQ5 4o-mini	40.00	47.00	51.67	72	37	2.1204
RQ5 MNC	45.71	31.07	58.54	101	42	50.2294
RQ6 4o-mini	66.67	69.00	62.61	33	16	1.8476
RQ6 MNC	62.96	43.04	51.67	92	52	39.8536
RQ7 4o-mini	40.68	5.001	52.38	114	57	2.8297
RQ7 MNC	44.07	44.07	52.79	190	72	66.1515
RQ8 4o-mini	19.67	29.00	51.67	35	15	1.2236
RQ8 MNC	26.23	26.45	51.67	148	80	85.4648
RQ9 4o-mini	41.94	49.00	61.67	81	34	2.4011
RQ9 MNC	40.32	44.64	53.18	81	54	37.2759
RQ10 4o-mini	58.97	70.00	73.4	60	30	3.4685
RQ10 MNC	51.28	51.95	52.46	86	46	65.4244

web content scraping, and multimedia searches. When integrated with LLMs, the Bocha Web Search API provides real-time information input to the LLMs. As LLMs perform tasks such as question answering, text generation, and dialogue, the Bocha API can extract the most up-to-date and relevant information from the internet based on specific query requests. This integration enhances the LLMs' ability to generate accurate and contextually appropriate responses, particularly for tasks requiring current or domain-specific knowledge.

5. Experimental result

5.1. Main result

The evaluation results presented in Tables 1–3 demonstrate the effectiveness of our proposed MNC framework across ten distinct network configuration scenarios. When compared to direct LLM prompting approaches, our method shows consistent and significant improvements in all evaluation metrics. Specifically, the framework achieves an average accuracy (ACC) improvement of 7.24%, indicating its superior performance in generating technically sound and precise network configurations.

The superior results can be attributed to several key aspects of our framework's design. The structured, multi-stage approach enables more precise handling of complex network requirements while maintaining strict adherence to technical specifications. The integration of verification and refinement mechanisms ensures higher consistency with industry best practices, and the knowledge-aware generation process produces configurations that better match real-world implementation requirements. These advantages are particularly evident in more complex scenarios involving VLAN configurations, inter-VLAN routing, and advanced features like port aggregation and RSTP implementation, where our framework maintains its performance edge over simpler prompting-based approaches.

5.2. Ablation study

The comprehensive evaluation presented in Table 4 provides valuable insights into the contribution of each component in our MNC framework. The complete framework demonstrates superior performance across all metrics compared to both the baseline LLM prompting approach and various ablated versions. Notably, the 23.86-point ACC

Table 2

Performance comparison between MNC and baseline based on DeepSeek.

	ACC	F1-score	BLEU	Tokens	Code length
RQ1 DS	45.71	22.07	51.67	194	18
RQ1 MNC	51.43	22.36	52.49	290	53
RQ2 DS	48.72	30.16	51.67	207	48
RQ2 MNC	53.85	63.64	51.67	35	19
RQ3 DS	46.94	36.22	52.55	127	25
RQ3 MNC	48.98	60.76	52.06	34	15
RQ4 DS	66.67	31.71	54.38	338	67
RQ4 MNC	56.41	59.46	51.67	110	52
RQ5 DS	42.86	22.22	51.67	278	58
RQ5 MNC	45.71	17.88	51.67	334	68
RQ6 DS	66.67	23.84	51.67	278	52
RQ6 MNC	66.67	72.00	51.67	32	16
RQ7 DS	23.73	21.87	51.67	162	20
RQ7 MNC	44.07	27.08	52.99	454	115
RQ8 DS	22.95	12.67	51.67	290	60
RQ8 MNC	19.67	29.27	51.67	25	10
RQ9 DS	27.42	20.61	51.67	187	32
RQ9 MNC	46.77	26.36	52.37	319	59
RQ10 DS	56.41	32.59	51.67	271	46
RQ10 MNC	56.41	60.27	53.38	93	46

Table 3

Performance comparison between MNC and baseline based on LLAMA-3.3-70B.

	ACC	F1-score	BLEU	Tokens	Code length
RQ1 LLAMA	14.29	8.06	51.67	171	13
RQ1 MNC	40.00	44.44	51.67	37	21
RQ2 LLAMA	7.69	8.22	51.67	15	10
RQ2 MNC	53.85	62.69	51.67	36	18
RQ3 LLAMA	18.37	21.95	51.67	10	8
RQ3 MNC	65.31	24.81	52.22	500	39
RQ4 LLAMA	58.97	38.02	58.73	74	42
RQ4 MNC	66.67	68.42	67.32	74	33
RQ5 LLAMA	31.43	27.16	51.67	33	17
RQ5 MNC	40.00	49.12	51.67	60	27
RQ6 LLAMA	25.93	10.77	51.67	193	13
RQ6 MNC	55.56	60.00	51.67	35	16
RQ7 LLAMA	13.56	17.02	51.67	26	10
RQ7 MNC	33.9	47.06	51.81	91	41
RQ8 LLAMA	6.56	7.34	51.67	19	12
RQ8 MNC	27.87	21.94	51.67	140	90
RQ9 LLAMA	29.03	23.84	51.67	152	25
RQ9 MNC	45.16	27.59	54.37	366	58
RQ10 LLAMA	38.46	22.06	51.67	139	29
RQ10 MNC	69.23	77.14	66.54	129	41

Table 4

Performance comparison between MNC and ablation models.

	ACC	F1-Score	BLEU	Token	Code length
MNC	51.35	45.32	53.34	125.9	52.3
Prompting LLM	27.49	20.46	51.67	145.0	18.1
- Critic Refinement	43.09	49.98	52.96	65.1	29.5
- Retrieval Tool	48.07	25.25	52.47	289.5	54.3
- Self-Consistency	42.90	49.91	52.18	60.9	30.2

advantage over direct LLM prompting (51.35 vs. 27.49) highlights the effectiveness of our integrated approach. The removal of the retrieval tool component results in a 3.28-point drop in ACC (from 51.35 to 48.07) and a more substantial 20.07-point decrease in F1-Score (from 45.32 to 25.25), indicating that external knowledge retrieval plays a crucial role in generating technically accurate configurations. This performance degradation suggests that relying solely on the model's parametric knowledge is insufficient for complex network configuration tasks that require up-to-date technical specifications and best

Table 5
Performance comparison of MNC using different iteration numbers.

Iteration num	ACC	F1-Score	BLEU	Token	Code length
0	48.45	24.31	52.65	288.8	56.0
1	44.05	22.75	52.31	265.4	43.5
2	50.26	22.65	52.21	336.3	53.6
4	47.93	19.09	52.17	385.5	54.3
5	50.81	20.48	52.55	372.2	58.6

practices. The critic refinement module proves particularly valuable, as its absence leads to an 8.26-point ACC reduction (from 51.35 to 43.09). This significant drop demonstrates how the iterative refinement process helps identify and correct configuration errors that might otherwise go unnoticed. Interestingly, the self-consistency mechanism shows comparable importance to the critic module, with its removal causing an 8.45-point ACC decrease. The similar token counts between these two ablated versions (65.1 vs. 60.9) indicate they both contribute to the framework's ability to maintain configuration coherence while optimizing output efficiency. The BLEU score remains relatively stable across variations (51.67–53.34), suggesting syntactic correctness is maintained even when other quality metrics fluctuate. These ablation studies collectively demonstrate that our framework's strength lies in the synergistic combination of its components, with each module addressing distinct aspects of the configuration generation process. The retrieval tool ensures technical accuracy, the critic module enables error correction, and the self-consistency mechanism maintains logical coherence - together enabling superior performance compared to both direct LLM prompting and partial implementations.

5.3. Analysis of computational cost

To evaluate the computational efficiency of our proposed MNC framework, we conducted a comprehensive analysis comparing it with baseline methods in terms of token consumption. As shown in Table 1, our multi-agent approach demonstrates distinct characteristics in computational resource utilization. The token-based evaluation methodology offers several advantages for assessing computational efficiency. Unlike inference time measurements that can be affected by hardware variations and network conditions, token counting provides a hardware-agnostic metric that directly reflects the computational workload of language model operations. This approach aligns with established practices in multi-agent system evaluation and enables meaningful cross-method comparisons. Our analysis reveals that MNC's token consumption is approximately 2.5 times higher than direct LLM prompting (59.2 vs. 148.1 tokens). This increase primarily stems from three key aspects of our framework: (1) the iterative refinement process involving multiple agents, (2) the knowledge retrieval and integration mechanism, and (3) the self-consistency verification steps. While this represents a significant computational overhead, the trade-off is justified by the substantial improvements in configuration quality demonstrated in our earlier results. Notably, the token efficiency varies across different configuration tasks. Basic configurations (e.g., VLAN setup) show a smaller token overhead, while complex tasks (e.g., RSTP implementation) demonstrate greater resource utilization. This scaling behavior indicates that our framework dynamically allocates computational resources based on task complexity.

5.4. Analysis of iteration number

In Section 3.2.3, we introduced the Iterative Refinement method, which involves multiple agents engaging in discussions and iteratively updating their data analysis results based on the outcomes of these discussions. In this section, we explore how many rounds of iteration are required for different problems to achieve satisfactory data analysis results. Table 5 presents the number of iterations used for the three

Table 6
Performance comparison when using different COT number.

	ACC	F1-Score	BLEU	Token	Code length
3 COT	51.35	45.32	53.34	125.9	52.3
1 COT	44.20	50.34	52.50	60.7	29.4

Table 7

Case study. Due to the limited space, we omit the detailed reasoning of each step.

Experiment Content: Using the Cisco Packet Tracer simulator, design, configure, and test a small local area network (LAN) to connect two PCs and enable communication between them via a switch.
Equipment Used: Switch: Cisco 2960 (1 unit) PCs: 2 units Cables: Straight-through cables
Generated Code:
Switch > enable; Switch # configure terminal; Switch (config) # hostname Switch1; Switch1 (config) # interface vlan 1; Switch1 (config-if) # ip address 192.168.1.254 255.255.255.0; Switch1 (config-if) # no shutdown; Switch1 (config-if) # exit; Switch1 (config) # interface range fa0/1–2; Switch1 (config-if-range) # switchport mode access; Switch1 (config-if-range) # switchport access vlan 1; Switch1 (config-if-range) # exit; Switch1 (config) # exit; Switch1 # write memory; PC1 > Desktop > IP Configuration; IP Address: 192.168.1.1; Subnet Mask: 255.255.255.0; Default Gateway: 192.168.1.254; PC2 > Desktop > IP Configuration; IP Address: 192.168.1.2; Subnet Mask: 255.255.255.0; Default Gateway: 192.168.1.254; PC1 > Desktop > Command Prompt; ping 192.168.1.2; PC2 > Desktop > Command Prompt; ping 192.168.1.1

research questions studied in this paper, along with the average number of iterations. The results in the table indicate that due to the complexity of the logic in RQ1, two rounds of iteration are needed to achieve a good self-consistent result, while the other two research questions (e.g., RQ2 and RQ3) can achieve satisfactory data analysis results with just one round of iteration. This phenomenon also demonstrates the effectiveness of the MNC we proposed.

5.5. Analysis of COT number

In Section 3.2, we employed a method where the model generates multiple chains of thought and then assesses their self-consistency to avoid producing erroneous data analysis results. Therefore, in this section, we explored the impact of using different numbers of COTs on the model's final performance. Table 6 illustrates the changes in the model's overall performance when using 1 and 3 COTs in a network configuration task. From Table 6, we observe that the model achieves its best performance with 3 COTs. This phenomenon suggests that an insufficient number of COTs fails to leverage the mutual enhancement effect of multiple results.

5.6. Case study

The case study presented in Table 7 provides a concrete demonstration of MNC's ability to generate network configurations. The example shows a complete LAN setup between two PCs through a Cisco 2960 switch, with all necessary configuration commands and verification steps. The generated configuration exhibits several important qualities that highlight the effectiveness of our approach. First, the configuration demonstrates technical accuracy through its correct use of Cisco IOS syntax and proper implementation of networking principles. Each command follows the expected format for switch configuration, including appropriate VLAN assignment and interface setup. The inclusion of IP addressing for both the switch management interface and end devices shows attention to practical implementation details. Second, the configuration shows operational completeness by covering all aspects needed for a working implementation. It includes not just the switch configuration, but also the PC network settings and verification commands. This end-to-end approach ensures the configuration can be directly implemented and tested, as evidenced by the successful ping tests between devices. Third, the example illustrates

the framework's ability to produce practically valid configurations. The generated commands were verified to work correctly in Cisco Packet Tracer, confirming they achieve the intended network connectivity. This practical validation is particularly important for network configuration tasks where theoretical correctness must translate to actual working implementations.

6. Conclusion

In this paper, we present Multi-agent based Network Configuration (MNC), a novel multi-agent framework that employ large language models for automated network configuration generation. The model integrates three core components: the Requirement Analysis Module for interpreting network specifications, the Configuration Generation Module for producing initial setups, and the Configuration Refinement Module for optimizing the final output. Through extensive experimentation on network configuration tasks, MNC demonstrates superior performance compared to state-of-the-art LLMs including LLaMa, GPT-4o-mini, and DeepSeek, achieving significant improvements in configuration accuracy, functional correctness, and syntactic precision. Comprehensive ablation studies validate the critical role of knowledge retrieval and iterative refinement mechanisms in ensuring configuration quality. While the multi-agent approach incurs greater computational overhead than direct LLM prompting, the substantial gains in configuration reliability and technical precision make it particularly valuable for enterprise network deployment scenarios where accuracy is paramount. These results establish MNC as an effective solution for automating complex network configuration tasks while maintaining adherence to networking best practices.

CRedit authorship contribution statement

Cui Wang: Writing. Huan Li: Experiments.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

References

- Aksitov, R., Miryosefi, S., Xiao Li, Z., Li, D., Babayan, S., Kopparapu, K., et al. (2023). Rest meets react: Self-improvement for multi-step reasoning LLM agent. *arXiv* [arXiv:2312.10003](https://arxiv.org/abs/2312.10003). URL <https://api.semanticscholar.org/CorpusID:266335848>.
- Cao, M., Shu, L., Yu, L., Zhu, Y., Wichers, N., Liu, Y., et al. (2024). Enhancing reinforcement learning with dense rewards from language model critic. In *Conference on empirical methods in natural language processing*. URL <https://api.semanticscholar.org/CorpusID:273901456>.
- Chan, C.-M., Chen, W., Su, Y., Yu, J., Xue, W., Zhang, S., et al. (2023). ChatEval: Towards better LLM-based evaluators through multi-agent debate. *arXiv* [arXiv:2308.07201](https://arxiv.org/abs/2308.07201). URL <https://api.semanticscholar.org/CorpusID:260887105>.
- Chen, X., Aksitov, R., Alon, U., Ren, J., Xiao, K., Yin, P., et al. (2023). Universal self-consistency for large language model generation. *arXiv* [arXiv:2311.17311](https://arxiv.org/abs/2311.17311). URL <https://api.semanticscholar.org/CorpusID:265498407>.
- Chen, X., Gao, S., Tao, C., Song, Y., Zhao, D., & Yan, R. (2018). Iterative document representation learning towards summarization with polishing. *arXiv* [arXiv:1809.10324](https://arxiv.org/abs/1809.10324). URL <https://api.semanticscholar.org/CorpusID:52879916>.
- Chen, L., Shang, S., Jensen, C. S., Yao, B., Zhang, Z., & Shao, L. (2019). Effective and efficient reuse of past travel behavior for route recommendation. In *Proceedings of the 25th ACM SIGKDD international conference on knowledge discovery & data mining* (pp. 488–498).
- Gao, S., Chen, X., Li, P., Ren, Z., Bing, L., Zhao, D., et al. (2018). Abstractive text summarization by incorporating reader comments. In *AAAI conference on artificial intelligence*. URL <https://api.semanticscholar.org/CorpusID:55461757>.
- Gao, S., Ren, Z., Zhao, Y. E., Zhao, D., Yin, D., & Yan, R. (2019). Product-aware answer generation in E-commerce question-answering. In *Proceedings of the Twelfth ACM International Conference on Web Search and Data Mining*. URL <https://api.semanticscholar.org/CorpusID:59158861>.
- Gao, S., Shi, Z., Zhu, M., Fang, B., Xin, X., Ren, P., et al. (2024). Confucius: Iterative tool learning from introspection feedback by easy-to-difficult curriculum. In M. J. Wooldridge, J. G. Dy, S. Natarajan (Eds.), *Thirty-eighth AAAI conference on artificial intelligence, AAAI 2024, thirty-sixth conference on innovative applications of artificial intelligence, IAAI 2024, fourteenth symposium on educational advances in artificial intelligence, EAAI 2024, February 20-27, 2024, Vancouver, Canada* (pp. 18030–18038). AAAI Press, URL <https://doi.org/10.1609/aaai.v38i16.29759>.
- Gao, Y., Xiong, Y., Gao, X., Jia, K., Pan, J., Bi, Y., et al. (2023). Vol. 2, Retrieval-augmented generation for large language models: A survey. *arXiv preprint* [arXiv:2312.10997](https://arxiv.org/abs/2312.10997).
- Gao, S., Zhang, Y., Wang, Y., Dong, Y., Chen, X., Zhao, D., et al. (2021). HeteroQA: Learning towards question-and-answering through multiple information sources via heterogeneous graph modeling. In *Proceedings of the fifteenth ACM international conference on web search and data mining*. URL <https://api.semanticscholar.org/CorpusID:245502395>.
- Guo, Z., Cheng, S., Wang, H., Liang, S., Qin, Y., Li, P., et al. (2024). StableToolBench: Towards stable large-scale benchmarking on tool learning of large language models. In *Annual meeting of the association for computational linguistics*. URL <https://api.semanticscholar.org/CorpusID:268363691>.
- Han, P., Yang, P., Zhao, P., Shang, S., Liu, Y., Zhou, J., et al. (2019). GCN-MF: disease-gene association identification by graph convolutional networks and matrix factorization. In *Proceedings of the 25th ACM SIGKDD international conference on knowledge discovery & data mining* (pp. 705–713).
- Han, S., Zhang, Q., Yao, Y., Jin, W., Xu, Z., & He, C. (2024). LLM multi-agent systems: Challenges and open problems. *arXiv* [arXiv:2402.03578](https://arxiv.org/abs/2402.03578). URL <https://api.semanticscholar.org/CorpusID:267499950>.
- Hong, S., Lin, Y., Liu, B., Wu, B., Li, D., Chen, J., et al. (2024). Data interpreter: An LLM agent for data science. *arXiv* [arXiv:2402.18679](https://arxiv.org/abs/2402.18679). URL <https://api.semanticscholar.org/CorpusID:268063292>.
- Hong, S., Zheng, X., Chen, J., Cheng, Y., Wang, J., Zhang, C., et al. (2023). Metagpt: Meta programming for multi-agent collaborative framework. *arXiv Preprint* [arXiv:2308.00352](https://arxiv.org/abs/2308.00352). URL <https://arxiv.org/abs/2308.00352>.
- Ishibashi, Y., & Nishimura, Y. (2024). Self-organized agents: A LLM multi-agent framework toward ultra large-scale code generation and optimization. *arXiv* [arXiv:2404.02183](https://arxiv.org/abs/2404.02183). URL <https://api.semanticscholar.org/CorpusID:268875891>.
- Joulin, A., Grave, E., Bojanowski, P., & Mikolov, T. (2016). Bag of tricks for efficient text classification. *arXiv* [arXiv:1607.01759](https://arxiv.org/abs/1607.01759). URL <https://api.semanticscholar.org/CorpusID:1210515>.
- Kojima, T., Gu, S. S., Reid, M., Matsuo, Y., & Iwasawa, Y. (2022). Large language models are zero-shot reasoners. In S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, & A. Oh (Eds.), *Advances in neural information processing systems 35: annual conference on neural information processing systems 2022, neurIPS 2022, New Orleans, LA, USA, November 28 - December 9, 2022*. URL http://papers.nips.cc/paper_files/paper/2022/hash/8bb0d291acd4ac06ef112099c16f326-Abstract-Conference.html.
- Li, C., Xue, M., Zhang, Z., Yang, J., Zhang, B., Wang, X., et al. (2025). START: Self-taught reasoner with tools. *arXiv preprint* [arXiv:2503.04625](https://arxiv.org/abs/2503.04625).
- Liu, S., Lu, Y., Chen, S., Hu, X., Zhao, J., Fu, T., et al. (2024). DrugAgent: Automating AI-aided drug discovery programming through llm multi-agent collaboration. *arXiv* [arXiv:2411.15692](https://arxiv.org/abs/2411.15692). URL <https://api.semanticscholar.org/CorpusID:274234223>.
- Papineni, K., Roukos, S., Ward, T., & Zhu, W.-J. (2002). Bleu: a method for automatic evaluation of machine translation. In *Proceedings of the 40th annual meeting of the association for computational linguistics* (pp. 311–318).
- Peng, L., Wang, Z., Yao, F., Wang, Z., & Shang, J. (2024). MetaIE: Distilling a meta model from LLM for all kinds of information extraction tasks. *arXiv* [arXiv:2404.00457](https://arxiv.org/abs/2404.00457). URL <https://api.semanticscholar.org/CorpusID:268819314>.
- Pu, Y., He, Z., Qiu, T., Wu, H., & Yu, B. (2024). Customized retrieval augmented generation and benchmarking for EDA tool documentation QA. *arXiv* [arXiv:2407.15353](https://arxiv.org/abs/2407.15353). URL <https://api.semanticscholar.org/CorpusID:271329372>.
- Shick, T., Dwivedi-Yu, J., Dessì, R., Raileanu, R., Lomeli, M., Hambro, E., et al. (2023). Toolformer: Language models can teach themselves to use tools. In A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, S. Levine (Eds.), *Advances in neural information processing systems 36: annual conference on neural information processing systems 2023, neurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023*. URL http://papers.nips.cc/paper_files/paper/2023/hash/d842425e4bf79ba039352da0f658a906-Abstract-Conference.html.
- Trivedi, H., Balasubramanian, N., Khot, T., & Sabharwal, A. (2023). Interleaving retrieval with chain-of-thought reasoning for knowledge-intensive multi-step questions. In A. Rogers, J. Boyd-Graber, & N. Okazaki (Eds.), *Proceedings of the 61st annual meeting of the association for computational linguistics (volume 1: long papers)* (pp. 10014–10037). Toronto, Canada: Association for Computational Linguistics, <https://doi.org/10.18653/v1/2023.acl-long.557>, URL <https://aclanthology.org/2023.acl-long.557>.
- Wang, H., Hao, S., Dong, H., Zhang, S., Bao, Y., Yang, Z., et al. (2024). Offline reinforcement learning for LLM multi-step reasoning. *arXiv* [arXiv:2412.16145](https://arxiv.org/abs/2412.16145). URL <https://api.semanticscholar.org/CorpusID:274965107>.

- Wei, J., Wang, X., Schuurmans, D., Bosma, M., Ichter, B., Xia, F., et al. (2022). Chain-of-thought prompting elicits reasoning in large language models. In S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, A. Oh (Eds.), *Advances in neural information processing systems 35: annual conference on neural information processing systems 2022, neurIPS 2022, new orleans, la, USA, November 28 - December 9, 2022*. URL http://papers.nips.cc/paper_files/paper/2022/hash/9d5609613524ecf4f15af0f7b31abca4-Abstract-Conference.html.
- Wu, Y., Sun, Z., Yuan, H., Ji, K., Yang, Y., & Gu, Q. (2024). Self-play preference optimization for language model alignment. arXiv [arXiv:2405.00675](https://arxiv.org/abs/2405.00675). URL <https://api.semanticscholar.org/CorpusID:269484698>.
- Yao, S., Zhao, J., Yu, D., Du, N., Shafraan, I., Narasimhan, K. R., et al. (2023). React: Synergizing reasoning and acting in language models. In *The eleventh international conference on learning representations, ICLR 2023, kigali, rwanda, May 1-5, 2023*. OpenReview.net, URL https://openreview.net/pdf?id=WE_vluYUL-X.
- Zeng, X., Liu, W., Huang, X., Wang, Z., Wang, L., Li, L., et al. (2025). ToolACE-R: Tool learning with adaptive self-refinement. arXiv preprint [arXiv:2504.01400](https://arxiv.org/abs/2504.01400).
- Zhang, X., Zhao, J. J., & LeCun, Y. (2015). Character-level convolutional networks for text classification. In *Neural information processing systems*. URL <https://api.semanticscholar.org/CorpusID:368182>.
- Zhao, Y., Xia, J., Liu, G., Su, H., Lian, D., Shang, S., et al. (2019). Preference-aware task assignment in spatial crowdsourcing. Vol. 33, In *Proceedings of the AAAI conference on artificial intelligence* (pp. 2629–2636).
- Zhou, D., Schärli, N., Hou, L., Wei, J., Scales, N., Wang, X., et al. (2023). Least-to-most prompting enables complex reasoning in large language models. In *The eleventh international conference on learning representations, ICLR 2023, kigali, rwanda, May 1-5, 2023*. OpenReview.net, URL <https://openreview.net/pdf?id=WZH7099tgfM>.